

Sistemas Distribuidos y Paralelos

Ingeniería en Computación



Introducción a los Sistemas Paralelos

Universidad Nacional de La Plata



Facultad de Informática





1

Concurrencia, Paralelismo y Sistemas Paralelos

Concurrencia y Paralelismo. Programa Concurrente y Programa Paralelo. Sistema Paralelo

2

Aplicaciones que usan Sistemas Paralelos

3

Estrategias para mejorar el rendimiento

Usar hardware más rápido: La ley de Moore, Surgimiento de los multicore, Otras leyes de crecimiento tecnológico, Los tres muros. Usar algoritmos más eficientes. Usar más hardware (Sistemas Paralelos)

4

Conceptos de paralelismo implícito

5

Paralelismo explícito

Arquitecturas paralelas y Sistemas Operativos. Modelos de programación. Comportamiento de las aplicaciones. Diseño. Lenguajes y bibliotecas. Desafíos en el desarrollo de aplicaciones paralelas. Métricas.

6

Cómputo de altas prestaciones (HPC)



1

Concurrencia, Paralelismo y Sistemas Paralelos

Concurrencia y Paralelismo

Programa Concurrente y Programa Paralelo

Sistema Paralelo

Concurrencia y Paralelismo

4

Concurrencia

Coexistencia de múltiples actividades en un mismo intervalo de tiempo.

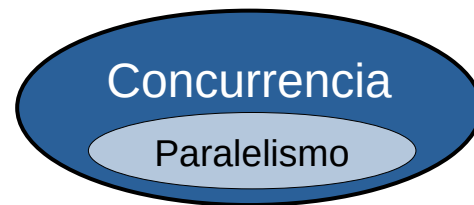
No es necesario que ocurran exactamente al mismo tiempo.

Actividades que se solapan temporalmente, avanzando de manera intercalada o compartiendo recursos.

Paralelismo

Caso particular de concurrencia donde las actividades ocurren realmente al mismo tiempo.

Existen múltiples recursos que permiten su ejecución simultánea.



Programa Concurrente y Programa Paralelo

5

Programa Concurrente

Múltiples hilos o procesos cuyas ejecuciones se solapan en el tiempo.

Pueden:

- Ser independientes
- Competir por recursos
- Cooperar

Requiere especificar comunicación y sincronización entre hilos o procesos.

No requiere varias unidades de procesamiento (*time slicing*).

Varias actividades avanzan en el mismo intervalo de tiempo. Es un concepto de software.

Programa Paralelo

Programa concurrente en el que varias tareas se ejecutan literalmente al mismo tiempo.

Requiere varias unidades de procesamiento.

Su **objetivo** es **mejorar el rendimiento**.

Varias actividades se ejecutan físicamente al mismo tiempo. Es un concepto de *hardware*.

Sistema Paralelo

6

Sistema Paralelo: Sistema organizado de *hardware* y *software* paralelos que permite ejecutar múltiples tareas de manera simultánea. Su objetivo es mejorar el rendimiento respecto a un algoritmo secuencial, ya sea reduciendo el tiempo de ejecución, aumentando la eficiencia energética o logrando ambos beneficios.



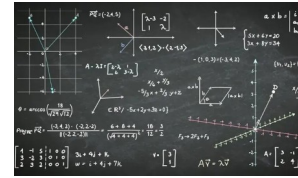
- Un **Sistema Paralelo** está compuesto por **dos componentes** principales y un **objetivo** fundamental:
 - **Software Paralelo (Algoritmo Paralelo):** Conjunto de **hilos o procesos** que **cooperan** en la resolución de un problema.
 - **Hardware Paralelo (Arquitectura Paralela):** Conjunto de **unidades de procesamiento** que permiten la ejecución simultánea de los hilos o procesos del software paralelo.
 - **Objetivo:** Mejorar el rendimiento respecto a la ejecución secuencial del problema.



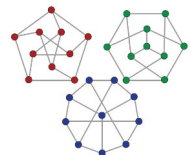
Aplicaciones que usan Sistemas Paralelos

8

- Mayormente, los sistemas paralelos se utilizan en:



- Procesamiento de grandes volúmenes de datos y análisis (Big Data, HPDA, IA).
- Algoritmos complejos y problemas combinatorios (TSP, optimización combinatoria, modelos econométricos).
- Simulación y predicción (clima, incendios, inundaciones).
- Álgebra lineal, bioinformática y procesamiento de imágenes.
- Modelado y simulación en física y química (simulación molecular, dinámica de fluidos).
- Criptografía y seguridad informática (cálculos intensivos de cifrado, blockchain).
- Robótica y vehículos autónomos.
- Computación financiera (simulación de riesgos, algoritmos de trading de alta frecuencia).
- Procesamiento de información en máquinas cuánticas (simulación cuántica, optimización y algoritmos híbridos cuántico-clásicos).



Estrategias para mejorar el rendimiento

Usar hardware más rápido

La ley de Moore

Surgimiento de los multicore

Otras leyes de crecimiento tecnológico

Los tres muros.

Usar algoritmos más eficientes.

Usar más hardware (Sistemas Paralelos)

Estrategias para mejorar el rendimiento

10

- En la vida cotidiana hay tres formas de mejorar el rendimiento:

Trabajar mas duro



Trabajar mas inteligentemente



Pedir ayuda y sumar voluntades



- Analogía con los **Sistemas Paralelos**:
 - Trabajar más duro = Usar hardware más rápido (**Simple**)
 - Trabajar más inteligentemente = Usar algoritmos optimizados (**Mayor esfuerzo**)
 - Pedir ayuda y sumar voluntades = Usar más hardware (**Mas complejo**)

Estrategias para mejorar el rendimiento

Usar hardware más rápido

La ley de Moore

Surgimiento de los multicore

Otras leyes de crecimiento tecnológico

Los tres muros.

Usar algoritmos más eficientes.

Usar más hardware (Sistemas Paralelos)

Usar hardware mas rápido

La ley de Moore

12

Ley de Moore (1965): El número de transistores que se pueden integrar en un circuito a costo constante se duplica aproximadamente cada 18–24 meses.

Moore's Law: The number of transistors on microchips doubles every two years

Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important for other aspects of technological progress in computing – such as processing speed or the price of computers.

Our World
in Data

Transistor count

50,000,000,000

10,000,000,000

5,000,000,000

1,000,000,000

500,000,000

100,000,000

50,000,000

10,000,000

5,000,000

1,000,000

500,000

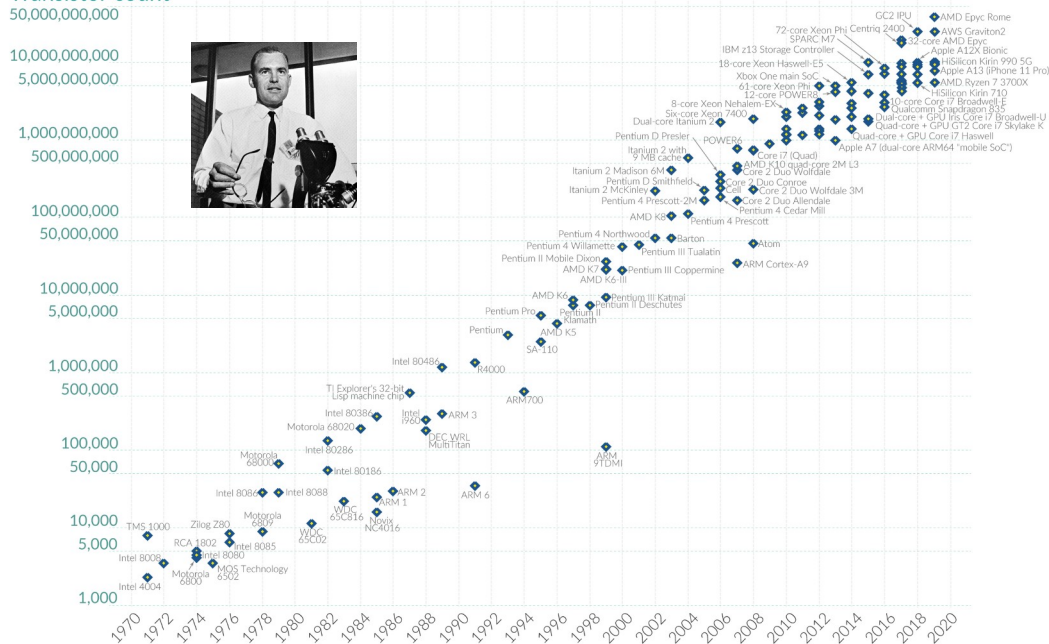
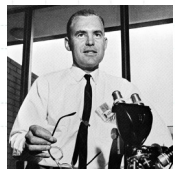
100,000

50,000

10,000

5,000

1,000



Data source: Wikipedia (wikipedia.org/wiki/Transistor_count)

OurWorldinData.org – Research and data to make progress against the world's largest problems.

Licensed under CC-BY by the authors Hannah Ritchie and Max Roser.

Mas transistores significa:

- Procesadores mas rápidos
- Aumento de la frecuencia de reloj
- Memorias caché mas grandes

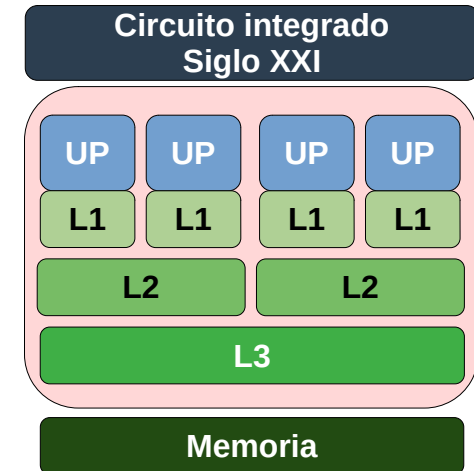
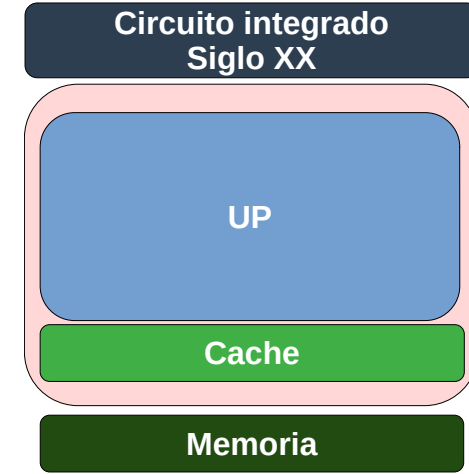
A principios del siglo XXI, esta tendencia llegaba a sus límites debido a problemas relacionados con el consumo eléctrico, la disipación térmica y la complejidad del diseño.

Usar hardware mas rápido

Surgimiento de los multicores

13

- **Siglo XX (décadas 1980-1990):** aumento de rendimiento incrementando la frecuencia de los procesadores.
 - Cada nueva generación de computadoras aproximadamente el doble de rápida que la anterior.
 - Un procesador una única unidad de procesamiento.
- **Siglo XXI:** Mayor frecuencia implica mayor consumo y a su vez mayor temperatura.
 - Calor imposible de disipar.
 - Frecuencia estabilizada en 3Ghz-4Ghz.
 - Surgimiento de los **multicores**: Un procesador varias unidades de procesamiento.



Usar hardware mas rápido

Otras leyes de crecimiento tecnológico

14

- Otras leyes, similares a la ley de Moore, relacionadas con comunicaciones:
 - **Ley de Gilder (Ley de la Banda Ancha):** La capacidad de las comunicaciones se triplica cada doce meses.
 - Impulsó los sistemas distribuidos y cloud
 - **Ley de Metcalfe:** El valor de una red es proporcional al cuadrado del número de usuarios.
 - Un solo dispositivo es inútil.
 - Su valor se incrementa con el número total de dispositivos de la red, por posibilidad de mayor comunicación.
 - **Ley de Reed:** La utilidad de redes grandes crece exponencialmente con el número de participantes.
 - Metcalfe: importancia del número de participantes.
 - Importancia del número posible de subgrupos que pueden formarse ($2^N - N - 1$ siendo N el número de participantes o posibles pares de conexiones).
 - Explica fenómenos de las redes sociales.

Usar hardware mas rápido

Los tres muros

15

- El hardware moderno se enfrenta a tres limitaciones (los tres muros):

Memory Wall	Power Wall	ILP Wall
• La velocidad de los procesadores creció más rápido que la velocidad de acceso a la memoria principal. • Latencia de memoria: principal problema de rendimiento en aplicaciones modernas. • Memorias caché mitigan el problema, pero no lo resuelven.	• Procesadores más rápidos consumen más energía. • No lineal: • Aumentar 73% la frecuencia (mas consumo) mejora solo el 13% el rendimiento. • Reducir la frecuencia en 13% reduce a la mitad el consumo energético. • Centros de computo limitados por la potencia eléctrica total instalada y la potencia de refrigeración/extracción	• Límites en el paralelismo a nivel de instrucción (ILP): por dependencias de datos y saltos condicionales • Explotar mas ILP se volvió complejo y costoso.

Estrategias para mejorar el rendimiento

Usar hardware más rápido

La ley de Moore

Surgimiento de los multicore

Otras leyes de crecimiento tecnológico

Los tres muros.

Usar algoritmos más eficientes.

Usar más hardware (Sistemas Paralelos)

Usar algoritmos más eficientes

17

- En algoritmos **secuenciales**:
 - **Reducir la complejidad del algoritmo:** Por ejemplo, pasar de un algoritmo $O(n^2)$ a uno $O(n \log n)$.
 - **Aprovechar la memoria caché:** A partir de explotar la localidad espacial y temporal.
 - **Minimizar espacio de almacenamiento:** Eliminar datos duplicados, usar estructuras compactas o representar la información de forma más eficiente.
- En algoritmos **paralelos**:
 - Minimizar la comunicaciones.
 - Minimizar la sincronización.
 - Optimización cómputo/comunicación (ocultamiento de latencia).

Estrategias para mejorar el rendimiento

Usar hardware más rápido

La ley de Moore

Surgimiento de los multicore

Otras leyes de crecimiento tecnológico

Los tres muros.

Usar algoritmos más eficientes.

Usar más hardware (Sistemas Paralelos)

Usar más hardware (Sistemas Paralelos)

Arquitecturas paralelas

19

- Hardware paralelo: Arquitecturas que permitan la ejecución simultanea de tareas.
- Dos tipos de arquitecturas paralelas:

Arquitecturas de memoria compartida	Arquitecturas de memoria distribuida
Multiprocesadores Multicores Manycores (GPUs, Xeon PHI)	Clusters Grid
Arquitectura Híbrida	
Combinación de arquitecturas de memoria compartida y memoria distribuida (Ejemplo: Cluster de Procesadores Multicore y GPUs)	



- Software paralelo: Algoritmos paralelos para beneficiarnos de las arquitecturas paralelas.
- Existen dos estrategias:
 - **Paralelismo implícito:** El control y el procesamiento es transparente al usuario.
 - *Transparencia: ocultar al programador los detalles de funcionalidad.*
 - **Paralelismo explícito:** El programador es responsable de implementar la lógica del programa paralelo.

La mayor parte de este curso esta orientado a Paralelismo Explícito

Usar mas hardware

Rendimiento

21

- El objetivo fundamental de un sistema paralelo es mejorar el rendimiento.
- Podemos analizar el rendimiento a partir de dos leyes:
 - **Ley de Amdahl:** *Para un problema de tamaño fijo, existe una fracción del programa inherentemente secuencial que limita la aceleración que puede obtenerse, independientemente del número de unidades de procesamiento que se utilicen.*
Como consecuencia, **utilizar mas unidades de procesamiento no siempre mejora el rendimiento, incluso puede empeorarlo.**
 - **Ley de Gustafson:** *Si contamos con mas unidades de procesamiento podemos resolver problemas más grandes.*
Como consecuencia, **utilizar mas unidades de procesamiento puede mejorar el rendimiento siempre y cuando el problema crezca lo suficiente.**



4

Conceptos de paralelismo implícito

Conceptos de paralelismo implícito

23

- Formas habituales de paralelismo implícito:
 - **Segmentación (pipelining):** Divide la ejecución de las instrucciones en etapas, permitiendo la ejecución simultánea de varias instrucciones.
 - **División funcional:** Múltiples unidades funcionales replicadas (por ejemplo, ALUs y FPUs) para la ejecución en paralelo, siempre que no existan dependencias.
- De esto se encarga el hardware de **bajo nivel** y las optimizaciones del compilador.
- **Aplicado en alto nivel, la transparencia facilita el desarrollo de software pero disminuye su eficiencia por *overhead* de gestión.**



Conceptos de paralelismo implícito

24

- Las ejecución de una instrucción en un procesador se realiza en varias etapas. Suponiendo un procesador de 5 etapas:

IF	Instruction Fetch	Búsqueda de la instrucción en memoria
ID	Instruction Decode	Decodificación de la instrucción y lectura de registros
E	Execution	Ejecución de la instrucción o cálculo de direcciones
MEM	Memory	Acceso a un operando en memoria de datos
WB	Write back	Escritura de resultado en un registro
NA	No Action	Sin acción

La ejecución de estas dos instrucciones se ejecutaría...



$a = b + c$
 $e = a - d$

LW R1, &10F0
LW R2, &1008
Add R3, R2, R1
LW R4, &1000
Sub R5, R3, R4
SW R5, &2000

IF	ID	E	MEM	WB
IF	ID	E	MEM	WB
IF	ID	E	MEM	WB
IF	ID	E	MEM	WB
IF	ID	E	MEM	WB
IF	ID	E	MEM	WB

Pero... no es una forma eficiente de aprovechar el hardware. Suponiendo que se ejecuta una etapa por ciclo, cada instrucción requiere 5 ciclos.



Conceptos de paralelismo implícito

Arquitectura Escalar

25

- Cada etapa usa hardware exclusivo. Se puede ejecutar cada etapa de manera que no interfiera en la ejecución del resto.



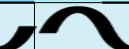
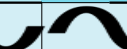
	Ciclo 0	Ciclo 1	Ciclo 2	Ciclo 3	Ciclo 4	Ciclo 5	Ciclo 6	Ciclo 7	Ciclo 8	Ciclo 9
LW R1, &10F0	IF	ID	E	MEM	WB					
LW R2, &1008		IF	ID	E	MEM	WB				
Add R3, R2, R1			IF	ID	E	MEM	WB			
LW R4, &1000				IF	ID	E	MEM	WB		
Sub R5, R3, R4					IF	ID	E	MEM	WB	
SW R5, &2000						IF	ID	E	MEM	WB

- El máximo paralelismo se alcanza cuando todas las etapas están activas, a partir de ese momento se ejecuta una instrucción en cada ciclo.
- **Arquitectura escalar:** arquitectura que ejecuta una instrucción por ciclo.

Conceptos de paralelismo implícito

Arquitectura Superescalar – Operaciones multiciclo

- **Arquitectura superescalar:** procesador que puede ejecutar más de una instrucción por ciclo.

	Ciclo 0	Ciclo 1	Ciclo 2	Ciclo 3	Ciclo 4	Ciclo 5	Ciclo 6	Ciclo 7	Ciclo 8	Ciclo 9
										
LW R1, &10F0	IF	ID	E	MEM	WB					
LW R2, &1008	IF	ID	E	MEM	WB					
Add R3, R2, R1		IF	ID	E	MEM	WB				
LW R4, &1000			IF	ID	E	MEM	WB			
Sub R5, R3, R4				IF	ID	E	MEM	WB		
SW R5, &2000					IF	ID	E	MEM	WB	

- **Operación multiciclo:** operación cuya etapa de ejecución demora más de un ciclo.
 - Operaciones de punto flotante.

	Ciclo 0	Ciclo 1	Ciclo 2	Ciclo 3	Ciclo 4	Ciclo 5	Ciclo 6	Ciclo 7	Ciclo 8	Ciclo 9
										
MUL R1, R2, R3	IF	ID	E	E	E	E	E	MEM	WB	
ADD R4, R1, R5		IF	ID	NA	NA	NA	NA	E	MEM	WB

Paralelismo explícito

Arquitecturas paralelas y Sistemas Operativos.

Modelos de programación.

Comportamiento de las aplicaciones.

Diseño.

Lenguajes y bibliotecas.

Desafíos en el desarrollo de aplicaciones paralelas.

Métricas.



Paralelismo explícito

28

- **Paralelismo explícito:** el programador es responsable de implementar la lógica del programa paralelo.
- Punto de partida:
 - Paralelizar un algoritmo secuencial.
 - Diseñar un algoritmo paralelo diferente al secuencial. La mejor solución puede diferir totalmente de la sugerida por los algoritmos secuenciales existentes:
 - Algoritmos que no tienen versión secuencial (Ej: **P**arallel **S**orting by **R**egular **S**ampling)

Paralelismo explícito

Aspectos a tener en cuenta

29

- Para desarrollar un algoritmo paralelo se deben considerar varios aspectos, en el siguiente orden:
 - 1) Arquitectura y Sistema Operativo
 - 2) Modelo de programación
 - 3) Comportamiento de las aplicaciones
 - 4) Diseño
 - 5) Lenguajes y bibliotecas
 - 6) Características del desarrollo de aplicaciones paralelas
 - 7) Métricas

Paralelismo explícito

Arquitecturas paralelas y Sistemas Operativos.

Modelos de programación.

Comportamiento de las aplicaciones.

Diseño.

Lenguajes y bibliotecas.

Desafíos en el desarrollo de aplicaciones paralelas.

Métricas.



- Antes de implementar un algoritmo paralelo debemos entender la arquitectura sobre la que vamos a ejecutarlo.
 - No podemos abstraernos del hardware.
- En la actualidad hay dos tipos de **arquitecturas paralelas**:
 - De **memoria compartida**.
 - De **memoria distribuida**.
- Podemos combinar ambos tipos de arquitectura en una **arquitectura híbrida**.

Paralelismo explícito

Sistemas operativos

32

- **Sistema operativo** para Sistemas Paralelos idealmente **dedicado**.
- No existen sistemas operativos desarrollados de "cero" para cómputo paralelo.
- Alternativas:
 - Adaptar distribuciones de sistemas operativos conocidas.
 - Utilizar distribuciones de sistemas operativos conocidas ya adaptadas con software paralelo.
 - Utilizar distribuciones de sistemas operativos más Robustas/Dedicadas.

SO Conocidos	SO Adaptados	SO Dedicados
• Linux: (Mayoritariamente) • FreeBSD/Solaris (Poco usadas) • Windows: NO!!! (no pensado para paralelismo)	• CentOS: basado en RedHat • Scientific Linux: basado en RedHat. Mantenido por el CERN.	• Rocks: basada en CentOS • FAI: basada en Debian • PelicanHPC: basado en Debian

Paralelismo explícito

Arquitecturas paralelas y Sistemas Operativos.

Modelos de programación.

Comportamiento de las aplicaciones.

Diseño.

Lenguajes y bibliotecas.

Desafíos en el desarrollo de aplicaciones paralelas.

Métricas.



Paralelismo explícito

Modelos de programación paralela

34

- El desarrollo de aplicaciones paralelas se basa en dos modelos de programación:

Modelo de memoria compartida	Modelo de memoria distribuida
Hilos/Procesos se comunican mediante la memoria. Herramientas de desarrollo: Pthreads OpenMP Otros (Cuda, OpenCL, Cilk, Sycl...)	Procesos se comunican mediante mensajes. Herramientas de desarrollo: PVM (en desuso) MPI Otros (RMI, Sockets, RPC...)
Modelo híbrido	
 Combinación de herramientas de desarrollo de ambos modelos (Ejemplo: MPI + OpenMP + CUDA)	

Paralelismo explícito

Modelos de programación paralela – Modelos de arquitectura paralela

35

- Los modelos de Programación Paralela (**Software**) y los modelos de Arquitectura Paralela (**Hardware**) son complementarios. Sin embargo, el modelo de programación paralela elegido depende del modelo de arquitectura paralela disponible.



		Software		
		Memoria Compartida (Ej: OpenMP)	Memoria Distribuida (Ej: MPI)	Híbrido (EJ: MPI + OpenMP)
Hardware	Memoria Compartida (Ej: multicore)	Trivial (Ej: OpenMP sobre multicore)	Posible (Ej: MPI sobre multicore)	Posible (extraño) (Ej: MPI + OpenMP sobre multicore)
	Memoria Distribuida (Ej: cluster monocore ¹)	(NO ADECUADO) Single System Image(SSl) Overhead	Trivial (Ej: MPI sobre cluster monocore)	Posible (muy extraño) (Ej: MPI + OpenMP sobre cluster monocore)
	Híbrido (Ej: cluster multicore)	(NO ADECUADO) Single System Image(SSl) Overhead	Posible (Ej: MPI sobre cluster multicore)	Trivial (Ej: MPI + OpenMP sobre cluster multicore)

¹Actualmente, los clusters monocore cayeron en desuso pero se mencionan para ejemplificar.

Paralelismo explícito

Arquitecturas paralelas y Sistemas Operativos.

Modelos de programación.

Comportamiento de las aplicaciones.

Diseño.

Lenguajes y bibliotecas.

Desafíos en el desarrollo de aplicaciones paralelas.

Métricas.



Paralelismo explícito

Comportamiento de las aplicaciones

37

- Debemos considerar el comportamiento de las aplicaciones a paralelizar:
 - **Intensivas en cómputo** (CPU Bound): llevan al límite la capacidad de la CPU.
 - La mayoría de las aplicaciones de cómputo científico.
 - **Intensivas en memoria** (Memory Bound): llevan al límite el sistema de memoria.
 - Ordenación
 - Búsquedas
 - **Intensivas en Entrada-Salida** (E/S Bound): llevan al límite el sistema de E/S.
 - Simulación
 - Big data
 - **Híbridos**: La aplicación presenta distintas fases intensivas.

Paralelismo explícito

Comportamiento de las aplicaciones

38

- Distintos eventos relacionados al cómputo requieren de un tiempo específico (Latencias).

Las magnitudes suelen ser despreciables y no permiten observar las diferencias reales.

¿Qué ocurre si escalamos ese tiempo?



Evento	Tiempo Real	Tiempo Escalado
1 ciclo de CPU	0.3 ns	1 s
Acceso a cache de Nivel 1	0.9 ns	3 s
Acceso a cache de Nivel 2	2.8 ns	9 s
Acceso a cache de Nivel 3	12.9 ns	43 s
Acceso a RAM	120 ns	6 min
E/S a SSD	50-150 μ s	2-6 días
E/S a HDD	1-10 ms	1-12 meses
Internet: San Francisco a Nueva York	40 ms	4 años
Internet: San Francisco a UK	81 ms	8 años
Internet: San Francisco a Australia	183 ms	19 años
Reboot de SO de VM	4 s	423 años
Reboot de SO físico	5 m	32 milenios



Paralelismo explícito

Arquitecturas paralelas y Sistemas Operativos.

Modelos de programación.

Comportamiento de las aplicaciones.

Diseño.

Lenguajes y bibliotecas.

Desafíos en el desarrollo de aplicaciones paralelas.

Métricas.



Paralelismo explícito

Diseño

40

- La **arquitectura** guía el modelo de programación, y junto con el sistema operativo y el tipo de aplicación a desarrollar, **guían el diseño** de la aplicación paralela.
- No existe una única estrategia de diseño. Pasos propuestos por Ian Foster¹:
 - **Descomposición o Particionado:** Identificar el trabajo que puede realizarse en paralelo, dividiéndolo en tareas.
 - **Comunicación:** Determinar los patrones de comunicación entre tareas.
 - **Aglomeración:** Combinar las tareas y las comunicaciones en tareas de mayor tamaño, para adecuarlas a las características del hardware paralelo disponible.
 - **Mapeo:** Asignar cada tarea a una unidad de procesamiento, buscando maximizar su utilización y minimizar los costos de comunicación.

¹*Libro: Designing and Building Parallel Programs - 1995*

Paralelismo explícito

Arquitecturas paralelas y Sistemas Operativos.

Modelos de programación.

Comportamiento de las aplicaciones.

Diseño.

Lenguajes y bibliotecas.

Desafíos en el desarrollo de aplicaciones paralelas.

Métricas.



Paralelismo explícito

Lenguajes de programación

42

- Podemos desarrollar aplicaciones paralelas con cualquier lenguaje que soporte la creación de hilos o procesos.
- Lenguajes de muy alto nivel, en especial interpretados, introducen *overhead* que impacta en el rendimiento.
- La mayoría de las aplicaciones paralelas están escritas en **C/C++ o Fortran**.
 - Wrappers de otros lenguajes traducen a código C (por ejemplo, numba o numpy en Python).

Pero C es Arcaico!!!
1972!!!

Siglo XXI → Lenguaje C ¿Donde?

Kernel SO:

Linux → Android
Windows
Etc.

Interpretes:

Java
Python
PHP
Etc.

Aplicaciones:

Office
Navegadores

Servicios:

WebServers
Apache
MailServers
AppServers
Tomcat
DBServers
MySQL
Postgress
MongoDB
Firewalls
Buscadores

Etc,etc y etc...

Paralelismo explícito

Arquitecturas paralelas y Sistemas Operativos.

Modelos de programación.

Comportamiento de las aplicaciones.

Diseño.

Lenguajes y bibliotecas.

Desafíos en el desarrollo de aplicaciones paralelas.

Métricas.



- Desafíos en el desarrollo de aplicaciones paralelas que no encontramos en aplicaciones secuenciales:

- No determinismo
- Sincronización
- Corrección y Depuración
- Fallas
- Comunicaciones
- Overhead de paralelismo
- Balance de carga
- Heterogeneidad
- Rendimiento y Escalabilidad
- Portabilidad
- Seguridad
- Dependencias de datos
- Granularidad de tareas
- Contención por recursos compartidos
- Asincronismo
- Consumo energético



Paralelismo explícito

Desafíos en el desarrollo de aplicaciones paralelas

45

No determinismo: Un programa paralelo es un programa concurrente y como tal, en distintas ejecuciones puede retornar distintos resultados.

Pero no todos estos resultados podrían ser válidos.

Sincronización: Para filtrar resultados no válidos debido a no determinismo es necesario sincronizar hilos o procesos.

También para evitar efectos no deseados (deadlocks, livelocks, idling, race conditions)

Mecanismo de sincronización: por exclusión mutua y/o por condición.

Corrección y Depuración: La corrección y depuración de un programa paralelo es muy compleja debido al no determinismo.

Se requiere un desarrollo cuidadoso y de una depuración del programa paso a paso.

Fallas: A mayor cantidad de *hardware* utilizado mayor posibilidad de falla.

Complejidad para detectar puntos de falla.

Programas de gran duración no deberían volver a ejecutarse de cero.

Importancia de aspectos como resiliencia, tolerancia y recuperación de fallas (*checkpoints*).

Paralelismo explícito

Desafíos en el desarrollo de aplicaciones paralelas

46

Comunicaciones: Los costos de comunicación por red de datos son generalmente altos con respecto al tiempo de cómputo.

Necesario minimizar el costo de las comunicaciones.

Importancia del ocultamiento de la latencia intercalando comunicaciones con cómputo.

Overhead de paralelismo: La creación de hilos o procesos, sincronización y comunicación agregan *overhead* que puede afectar los beneficios del paralelismo.

Necesario minimizar este *overhead*.

Balance de carga: Distribuir equilibradamente el trabajo entre las unidades de procesamiento.

Evitar unidades de procesamiento ociosas o sobrecargadas.

Existen aplicaciones dinámicas que provocan desbalances naturalmente y necesitan balancear la carga en tiempo de ejecución.

Heterogeneidad: En varios niveles: *Hardware*, Sistemas Operativos, Redes, Herramientas etc.

Se generan problemas relacionados a Compilación y Balance de carga.

Paralelismo explícito

Desafíos en el desarrollo de aplicaciones paralelas

47

Rendimiento y Escalabilidad: El objetivo es obtener el máximo rendimiento posible respecto al algoritmo secuencial. Se usan métricas para cuantificar ese rendimiento (Speedup, Eficiencia). La solución paralela debería ser escalable: incrementar el número de unidades de procesamiento y la carga de trabajo del problema sin perder rendimiento.

Portabilidad: Es fácil lograr portabilidad del código entre distintas arquitecturas. Es difícil lograr portabilidad de rendimiento, es decir lograr que el programa alcance en diferentes arquitecturas un rendimiento similar. Rediseño de algoritmos: portar un algoritmo paralelo desarrollado para multicores hacia una GPU.

Seguridad: Políticas de seguridad necesarias para evitar ataques que usen la potencia de un sistema paralelo para actividades “ilícitas”.

Dependencias de datos: Algoritmos que deben ejecutar pasos que requieren de la ejecución previa de otros pasos, limitando el paralelismo.

Paralelismo explícito

48

Desafíos en el desarrollo de aplicaciones paralelas

Granularidad de tareas: La elección de la granularidad (tamaño de las unidades de trabajo) afecta la eficiencia.

Tareas demasiado pequeñas pueden generar *overhead*.

Tareas demasiado grandes pueden provocar desbalance de carga.

Contención de recursos compartidos: El acceso concurrente a recursos compartidos (memoria, caché, buses) puede generar contención, afectando el rendimiento.

Asincronismo: En sistemas paralelos o distribuidos, no existe un “reloj universal” compartido entre todos los procesos. Cada nodo de un cluster o procesador puede tener su propio reloj, que puede avanzar a velocidades ligeramente diferentes o estar desincronizado respecto a otros. Por esto, la ocurrencia de un evento puede ser percibida en distintos instantes y medir tiempos absolutos puede ser poco confiable.

Consumo energético: En sistemas de gran escala, la eficiencia energética puede ser un factor a considerar.

Paralelismo explícito

Arquitecturas paralelas y Sistemas Operativos.

Modelos de programación.

Comportamiento de las aplicaciones.

Diseño.

Lenguajes y bibliotecas.

Desafíos en el desarrollo de aplicaciones paralelas.

Métricas.



Paralelismo explícito

Métricas

50

- Métricas para evaluar cuán bueno es nuestro algoritmo paralelo.
- Entre las métricas mas importantes encontramos:
 - Speedup
 - Eficiencia
- A partir de estas métricas analizamos:
 - La ley de Amdhal.
 - La ley de Gustafson-Barsis.
 - Escalabilidad: comportamiento del algoritmo al agregar mas unidades de procesamiento y aumentar la carga de trabajo.



6

Cómputo de altas prestaciones (HPC)

Cómputo de altas prestaciones (HPC)

52

- Los sistemas paralelos son parte de un concepto mas amplio:

High Performance Computing (HPC)

(Cómputo de altas prestaciones o computación de alto rendimiento)

HPC: Práctica de sumar potencia de cómputo con el fin de alcanzar mayor rendimiento en la ejecución de programas que resuelven problemas complejos o de gran tamaño de la ciencia, ingeniería o negocios.

- **HPC** abarca el estudio de distintas áreas:

